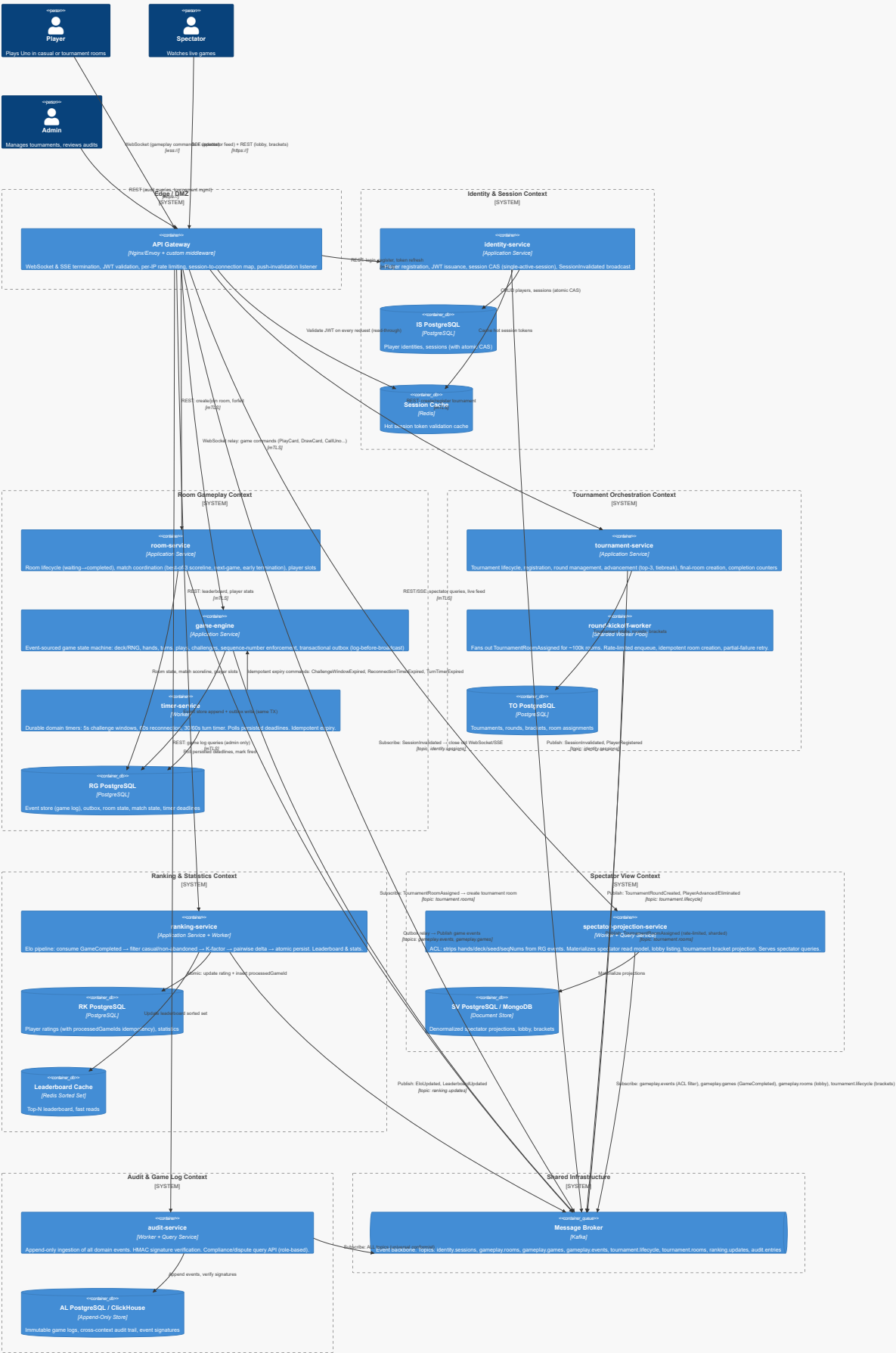


2. Container View — C4 Level 2

All major runnable components (services, workers, gateways, brokers, stores), their interactions, and trust boundaries. Corresponds to a C4 Container diagram.

2.1 Container Diagram

UnoArenas - Container Diagram



2.2 Kafka Topic Layout

Topic	Partitioning Key	Producers	Consumers	Purpose
identity.sessions	playerId	identity-service	api-gateway, game-engine, tournament-service, ranking-service, audit-service	Session lifecycle events: <code>PlayerRegistered</code> , <code>SessionEstablished</code> , <code>SessionInvalidated</code> , <code>PlayerSuspended</code> , <code>PlayerBanned</code>
gameplay.rooms	roomId	room-service	tournament-service, spectator-projection-service, audit-service	Room lifecycle: <code>RoomCreated</code> , <code>PlayerJoinedRoom</code> , <code>PlayerLeftRoom</code> , <code>RoomFilled</code> , <code>MatchGameCompleted</code> , <code>MatchCompleted</code> , <code>RoomCompleted</code>
gameplay.games	gameId	game-engine (via outbox relay)	room-service, ranking-service, spectator-projection-service, audit-service	Game-level completion: <code>GameCompleted</code>
gameplay.events	gameId	game-engine (via outbox relay)	room-service, tournament-service, spectator-projection-service, ranking-service, audit-service	All gameplay state-change events: <code>CardPlayed</code> , <code>CardDrawn</code> , <code>TurnAdvanced</code> , <code>TurnPassed</code> , <code>PlayerForfeited</code> , <code>PlayerDisconnected</code> , <code>PlayerReconnected</code> , WDF/Uno challenge events, etc. — high volume

Topic	Partitioning Key	Producers	Consumers	Purpose
tournament.lifecycle	tournamentId	tournament-service	spectator-projection-service, ranking-service, audit-service	TournamentCreated, RegistrationOpened, TournamentStarted, TournamentRoundCreated, PlayerAdvanced, PlayerEliminated, FinalRoomCreated, AllMatchesInRoundCompleted, TournamentCompleted
tournament.rooms	roomId	round-kickoff-worker	room-service	TournamentRoomAssigned — high burst at round start
ranking.updates	playerId	ranking-service	audit-service	EloUpdated, LeaderboardUpdated, PlayerStatisticsUpdated

Ordering guarantees: Partitioning by aggregate ID (`gameId`, `roomId`, etc.) ensures total ordering of events within a single aggregate. Cross-aggregate ordering is eventual.

2.3 Component Scaling Summary

Component	Horizontal?	Partition/Shard Key	Singleton Risk	Notes
api-gateway	Yes	Stateless (session map in Redis)	None	Scale with connection count
identity-service	Yes	Stateless	None	DB is bottleneck; cacheable reads
room-service	Yes	roomId	None	Partition affinity optional
game-engine	Yes	gameId	None	Event-sourced; any instance can rebuild from log
timer-service	Yes	Deadline bucket (time shard)	None	Multiple instances poll disjoint deadline ranges
tournament-service	Yes	tournamentId	Low risk (few concurrent tournaments)	Completion counter needs atomic increment
round-kickoff-worker	Yes	Round partition (player range)	None	Key scaling lever for first-round surge

Component	Horizontal?	Partition/Shard Key	Singleton Risk	Notes
ranking-service	Yes	Consumer group partitions	None	Scales with GameCompleted throughput
spectator-projection-service	Yes	Consumer group partitions	None	Scales with spectator query load + event volume
audit-service	Yes	Consumer group partitions	None	Append-only; scales with total event volume